INSTALLAZIONE / RASPBERRY PI 3B+

Configurare la Raspberry Pi con il backend Firebase

Guida passo per passo per installare, avviare e verificare il controllo locale

Questa guida installa il nuovo backend Python sulla Raspberry Pi, abilita i sensori SPI e configura l'avvio automatico come servizio systemd. Firebase sincronizza stato, planning e comandi; la regolazione dei relè e la configurazione attiva restano sulla Raspberry Pi. Dopo un riavvio, il backend rilegge il file locale config.json anche se la rete non è disponibile.

Prima di iniziare

- Raspberry Pi 3B+ con Raspberry Pi OS e accesso a un account con sudo.
- Connessione SSH oppure tastiera e schermo per i comandi sul Pi.
- Due moduli MAX6675 con termocoppie K e due moduli relè compatibili con la logica a 3,3 V.
- API key, URL del Realtime Database, email, password e UID dell'utente Firebase dedicato.
- I file del backend aggiornato sul PC. Questa procedura li trasferisce con SCP: non richiede un push su Git.

Due regole operative

Sicurezza elettrica. Tieni scollegati i carichi di rete durante la prima verifica. Il backend inizializza i relè su OFF, ma può attivarli dopo i primi campioni di temperatura. Completa il cablaggio dei circuiti a tensione di rete solo con adeguate protezioni e competenza elettrica.

Uso senza rete. Il servizio parte senza aspettare la connessione Internet. Se Firebase non è raggiungibile, il controllo continua con l'ultimo config.json locale e la sincronizzazione riprova in background.

Credenziali. Il segreto cifrato systemd deve essere creato sulla Raspberry Pi che eseguirà il servizio. Non copiare il file cifrato da Windows o da un'altra installazione Linux.

1 Collega sensori e relè

Scollega l'alimentazione prima di cablare. Il backend usa la numerazione BCM: GPIO17 e GPIO27 identificano le linee logiche, mentre i numeri fisici indicano la posizione sul connettore a 40 pin.

Collegamenti dei due MAX6675

Segnale modulo	GPIO Raspberry	Pin fisico	Collegamento
VCC	—	1	3V3; verifica che il breakout sia compatibile con la logica a 3,3 V
GND	—	6	Massa comune
SCK	`GPIO11 / SCLK`	23	Comune ai due moduli
SO	`GPIO9 / MISO`	21	Comune ai due moduli
CS del CH1	`GPIO8 / CE0`	24	Chip select del primo sensore
CS del CH2	`GPIO7 / CE1`	26	Chip select del secondo sensore

Ingressi dei moduli relè

Cd Canale	GPIO BCM	Pin fisico	Ingresso modulo
CH1	`GPIO17`	11	IN / SIG del modulo relè 1
CH2	`GPIO27`	13	IN / SIG del modulo relè 2

Il driver relè deve accettare il livello logico a 3,3 V del Raspberry Pi. Non collegare bobine, carichi o contatti di rete direttamente ai GPIO. Le uscite GPIO del Pi non tollerano 5 V: alimentare un breakout a 5 V solo se il suo circuito garantisce livelli sicuri verso il Raspberry.

Il codice apre SPI bus 0, device 0 per CH1 e device 1 per CH2. Legge GPIO17 e GPIO27 come uscite relè e usa la logica attiva alta (RELAY_ACTIVE_LOW = False). Se il tuo modulo è attivo basso, non cambiare questa impostazione senza aver prima verificato il comportamento con i carichi scollegati.

2 Prepara Raspberry Pi OS

01 Accedi con l’utente che eseguirà il servizio

Nell’esempio l’utente è pi e la cartella del progetto è /home/pi/rpi_thermostat. Se il tuo utente ha un altro nome, sostituiscilo in tutti i percorsi e nel file systemd.

```
whoami
hostname -I
systemd --version | head -n 1
systemd-creds --version
```

systemd-creds e le credenziali LoadCredentialEncrypted sono disponibili da systemd 250. Se la versione installata è inferiore a 250, fermati prima del passaggio credenziali e aggiorna Raspberry Pi OS a una versione supportata.

02 Aggiorna l’indice e installa i pacchetti Python

```
sudo apt update
sudo apt install -y python3 python3-requests python3-gpiozero python3-spidev
```

Questi pacchetti forniscono Python, richieste HTTPS Firebase, GPIO Zero e accesso SPI. Non serve Flask per il nuovo backend.

03 Abilita SPI

```
sudo raspi-config
```

Nel menu seleziona 3 Interface Options → I4 SPI → Yes, poi Finish. Riavvia il Pi se richiesto. Raspberry Pi OS abilita l’interfaccia e carica il modulo SPI al boot.

```
sudo reboot
# Dopo il riavvio
ls -l /dev/spidev0.0 /dev/spidev0.1
```

Entrambi i file /dev/spidev0.0 e /dev/spidev0.1 devono comparire. Se uno manca, ricontrolla l’abilitazione SPI e riavvia.

3 Trasferisci il backend e conserva il planning

Prima di avviare il nuovo processo, arresta l’eventuale vecchio backend: due processi non devono controllare gli stessi GPIO. Se avevi già config.json sulla Raspberry Pi, mantienilo nella cartella del progetto; contiene l’ultima configurazione e il planning locale.

Arresta e salva la configurazione esistente

```
sudo systemctl list-units --type=service --all | grep -i thermostat
# Se trovi il vecchio servizio, fermalo usando il suo nome effettivo
sudo systemctl stop NOME_SERVIZIO_PRECEDENTE
cd /home/pi/rpi_thermostat
test ! -f config.json || cp -a config.json "config.json.pre-firebase.$(date +%Y%m%d-%H%M%S).bak"
```

Sostituisci NOME_SERVIZIO_PRECEDENTE solo se il servizio esiste. Se il vecchio backend era stato avviato a mano, interrompi quel processo prima di proseguire. Non copiare un nuovo config.json sopra a quello già presente.

Invia i file dal PC Windows senza Git

Apri PowerShell nella cartella locale del progetto. Sostituisci l’indirizzo 192.168.1.50 con quello mostrato da hostname -I; sostituisci pi se usi un altro account. SSH deve essere abilitato sul Raspberry.

```
ssh pi@192.168.1.50 "mkdir -p /home/pi/rpi_thermostat"
scp .\rpi_thermostat_backend_main.py .\firebase_client.py .\setup_firebase_credentials.py
.\thermostat.service.example pi@192.168.1.50:/home/pi/rpi_thermostat/
```

I file essenziali sono il backend, il client Firebase, lo script credenziali e l’esempio del servizio. Il file config.json resta sul Pi. Nessuna modifica viene inviata a Git.

```
# Sul Raspberry Pi
cd /home/pi/rpi_thermostat
ls -l rpi_thermostat_backend_main.py firebase_client.py setup_firebase_credentials.py
thermostat.service.example
```

Come viene recuperato config.json

Al primo avvio, se il file non esiste, il backend crea una configurazione iniziale. Negli avvii successivi legge e normalizza config.json prima di iniziare il controllo. Salva le modifiche con scrittura temporanea e rinomina atomica. Se il file è danneggiato, si ferma senza sostituirlo con valori predefiniti: conserva la copia e correggilo prima di riavviare.

4 Configura le credenziali Firebase

Usa lo stesso account Linux indicato da User= nel servizio (nell'esempio pi). Non eseguire lo script con sudo: il percorso cifrato deve finire nella home dell'utente previsto.

```
cd /home/pi/rpi_thermostat
python3 setup_firebase_credentials.py
```

Inserisci Web API key, URL del Realtime Database, email, password e UID dell'utente Firebase. La password viene richiesta senza mostrarla sul terminale. La Web API key identifica il progetto; le regole RTDB e l'account dedicato controllano l'accesso.

Lo script crea ~/.config/rpi_thermostat/firebase_config.cred cifrato tramite systemd-creds. Il testo in chiaro viene scritto temporaneamente con permessi restrittivi e poi rimosso dallo script; non creare o trasferire un JSON con la password.

```
systemd-creds --version
stat -c '%a %n' ~/.config/rpi_thermostat ~/.config/rpi_thermostat/firebase_config.cred
```

I permessi attesi sono 700 sulla cartella e 600 sul file .cred. La chiave systemd è legata all'installazione locale: ricrea la credenziale sul Pi dopo una reinstallazione del sistema o una sostituzione del dispositivo.

5 Installa il servizio systemd

```
sudo cp thermostat.service.example /etc/systemd/system/thermostat.service
sudo nano /etc/systemd/system/thermostat.service
```

Controlla e modifica questi valori se non usi l'utente pi o il percorso d'esempio:

Direttiva	Valore d'esempio	Deve corrispondere a
User	pi	Utente proprietario delle credenziali cifrate
WorkingDirectory	/home/pi/rpi_thermostat	Cartella con i file Python e config.json
ExecStart	/usr/bin/python3 /home/pi/rpi_thermostat/rpi_thermostat_backend_main.py	Interprete e percorso reale del backend
SupplementaryGroups	gpio spi	Gruppi che autorizzano GPIO e SPI
LoadCredentialEncrypted	/home/pi/.config/rpi_thermostat/firebase_config.cred	Percorso prodotto dal setup credenziali

Il file d'esempio non aspetta network-online.target: il servizio parte anche senza rete e il backend continua il controllo locale. systemd consegna al processo la credenziale decifrata in una directory temporanea di sola lettura.

```
sudo systemd-analyze verify /etc/systemd/system/thermostat.service
```

6 Avvia e verifica il backend

Per il primo avvio lascia scollegato il carico comandato dai relè. Controlla che i moduli siano correttamente cablati e che nessun relè possa alimentare una resistenza o un carico pericoloso durante la prova.

```
sudo systemctl daemon-reload
sudo systemctl enable --now thermostat
sudo systemctl status thermostat --no-pager
```

Il primo comando ricarica le unità systemd; il secondo abilita l'avvio a ogni boot e avvia subito il servizio. Lo stato deve risultare active (running).

Leggi i log

```
sudo journalctl -u thermostat -n 80 --no-pager
sudo journalctl -u thermostat -f
# Premi Ctrl+C per uscire dal log in tempo reale
```

Cerca l'avvio del backend e l'assenza di errori su SPI, GPIO o credenziali. Quando Firebase è disponibile, il nodo thermostat/status viene aggiornato ogni secondo. Il frontend considera il Raspberry online quando il timestamp ha meno di 10 secondi.

Controlla file locale e cloud

```
sudo systemctl is-enabled thermostat
sudo systemctl is-active thermostat
ls -l /home/pi/rpi_thermostat/config.json
cat /home/pi/rpi_thermostat/config.json
```

- Il servizio risponde enabled e active.
- config.json esiste accanto al backend. Se esisteva già, contiene ancora i setpoint e i planning precedenti.
- In Firebase Console, thermostat/status ha updated_at recente e config_revision coerente.
- Il frontend web mostra TERMOSTATO ONLINE entro circa 10 secondi dalla pubblicazione dello stato.

Ora di sistema e planning

Il backend confronta gli intervalli del planning con l'orologio locale del Raspberry Pi. Imposta il fuso Europe/Rome e verifica che l'ora sia sincronizzata quando la rete è disponibile:

```
sudo raspi-config
# 5 Localisation Options -> L2 Timezone -> Europe -> Rome
timedatectl status
```

config.json conserva il planning anche senza corrente o rete. Senza un RTC esterno, però, l'ora esatta dopo una lunga interruzione di corrente avviata senza rete non è garantita: la pianificazione può essere valutata rispetto a un orario errato finché il Pi non si sincronizza. Per rispettare gli orari anche in questa condizione, aggiungi un modulo RTC compatibile.

Verifica il riavvio senza rete

Dopo aver verificato i carichi in sicurezza, riavvia il Pi e controlla che thermostat sia active. Per provare il funzionamento offline, togli solo la connessione di rete, senza togliere alimentazione ai circuiti di controllo: il frontend mostrerà offline, mentre il backend continuerà con config.json. Ripristinata la rete, la sincronizzazione Firebase riparte automaticamente.

7 Risolvi i problemi più comuni

Sintomo	Controllo	Azione
systemd-creds mancante o versione < 250	systemd-creds --version	Aggiorna Raspberry Pi OS a una versione con systemd 250 o successivo.
Il servizio non parte	sudo journalctl -u thermostat -n 80 --no-pager	Correggi User, WorkingDirectory, ExecStart e il percorso LoadCredentialEncrypted.
Permission denied su GPIO/SPI	id pi; ls -l /dev/spidev0.*	Verifica gruppi gpio e spi e la direttiva SupplementaryGroups; riavvia il servizio.
File /dev/spidev0.0 o .1 assente	ls -l /dev/spidev0.0 /dev/spidev0.1	Abilita SPI con raspi-config e riavvia il Pi.
Credenziale non leggibile	stat -c '%a %n' ~/.config/rpi_thermostat/firebase_config.cred	Crea il .cred come utente del servizio e aggiorna il percorso nella unit.
Frontend offline ma servizio attivo	Apri thermostat/status e controlla updated_at	Controlla Wi-Fi/ethernet, le regole Firebase e i log di autenticazione. L'offline cloud non ferma il controllo locale.
Il backend si ferma su config.json	Leggi l'errore nei log e conserva il file	Ripristina la copia di sicurezza valida; non cancellare config.json se contiene il planning da recuperare.

Aggiorna il codice senza toccare i planning

Per una nuova versione, arresta il servizio, copia i file Python aggiornati nella stessa cartella, poi riavvia. Non sostituire config.json e non rigenerare il .cred se account e sistema sono invariati.

```
sudo systemctl stop thermostat
# Da Windows: ripeti il comando scp della pagina precedente
sudo systemctl start thermostat
sudo systemctl status thermostat --no-pager
```

Il file config.json resta sul disco locale e viene caricato prima del controllo. Conserva la copia di sicurezza in un luogo protetto finché non hai verificato il servizio.

Abilita il pulsante di spegnimento (facoltativo)

Il backend invoca sudo shutdown now quando riceve il comando web. Se il tuo utente di servizio non ha già un'autorizzazione sudo senza password per questo comando, puoi concedere solo lo spegnimento al posto di abilitare sudo generale.

```
command -v shutdown
sudo visudo -f /etc/sudoers.d/rpi-thermostat-shutdown
# Nel file, sostituisci pi e il percorso con i valori reali:
pi ALL=(root) NOPASSWD: /usr/sbin/shutdown now
sudo chown root:root /etc/sudoers.d/rpi-thermostat-shutdown
sudo chmod 0440 /etc/sudoers.d/rpi-thermostat-shutdown
sudo visudo -cf /etc/sudoers.d/rpi-thermostat-shutdown
```

Se command -v shutdown restituisce un percorso diverso, usa quel percorso nella regola. La sezione è facoltativa: il controllo termico locale funziona anche se non abiliti lo spegnimento remoto.

Riferimenti tecnici

Raspberry Pi Documentation · Abilitare SPI · Consultato il 27 ottobre 2026

Raspberry Pi Documentation · Fuso orario · Consultato il 27 ottobre 2026

Raspberry Pi Documentation · Pin GPIO e livelli logici · Consultato il 27 ottobre 2026

Raspberry Pi Documentation · Orologio RTC · Consultato il 27 ottobre 2026

Analog Devices · Datasheet MAX6675 · Consultato il 27 ottobre 2026

systemd.exec · credenziali per servizi · Consultato il 27 ottobre 2026

systemd-creds · encrypt e versioni supportate · Consultato il 27 ottobre 2026